Structure of Programming Languages

Lexical Analysis

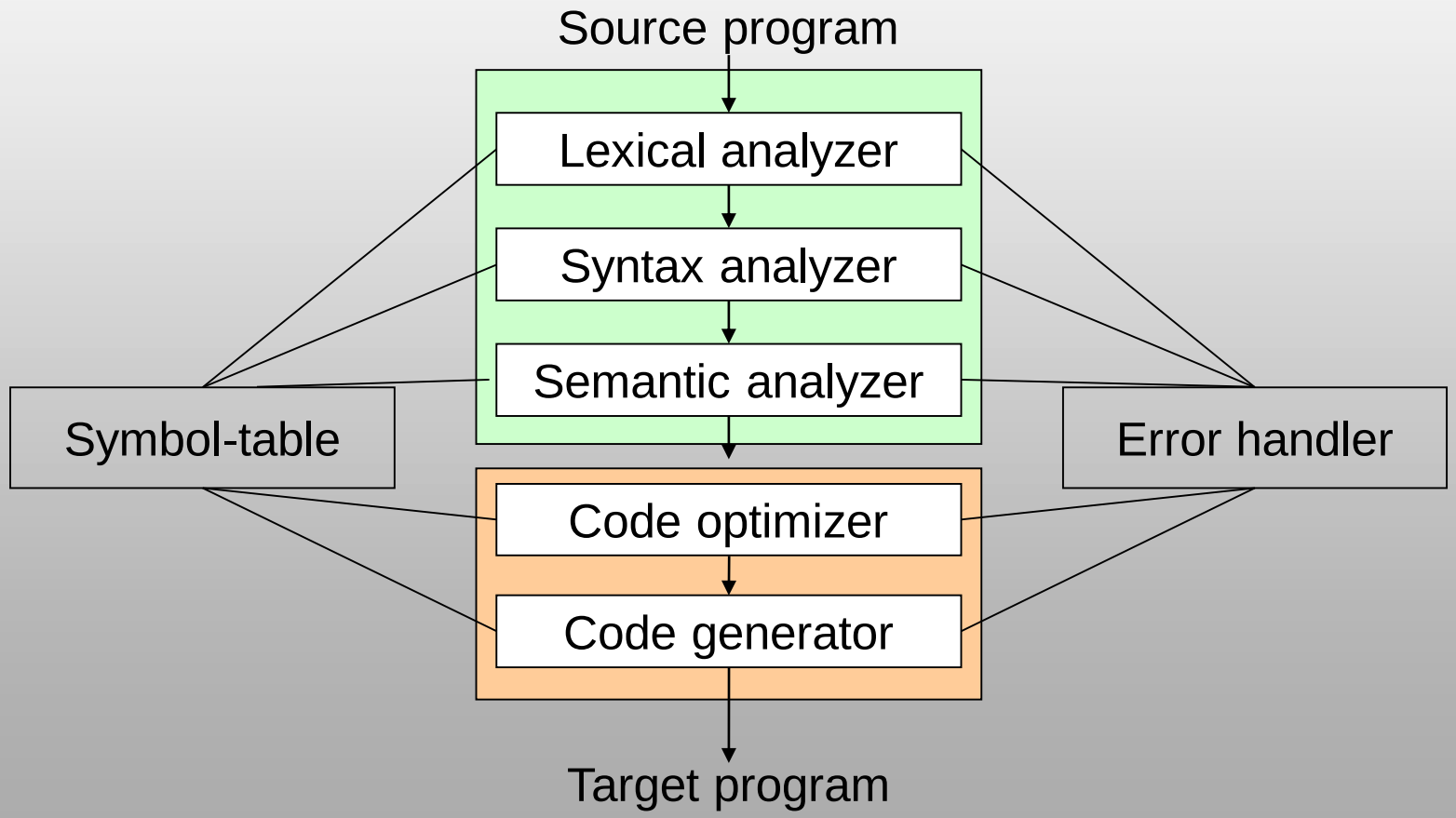
What is a compiler?

A program that reads a program written in one language and translates it into another language.



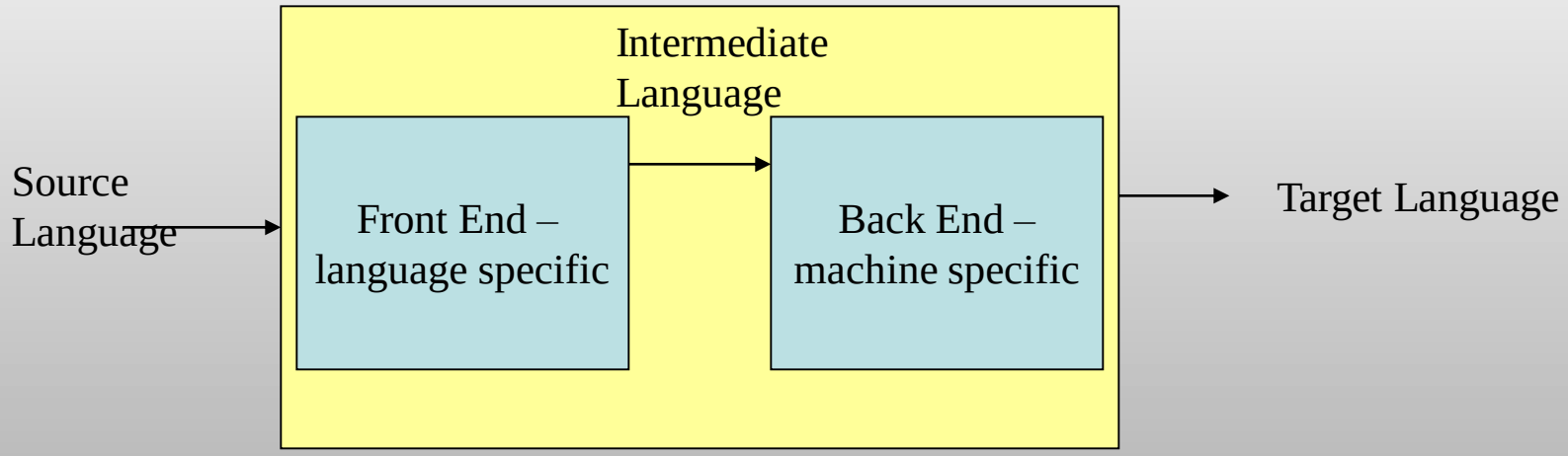
Traditionally, compilers go from high-level languages to low-level languages.

Compiler Overview



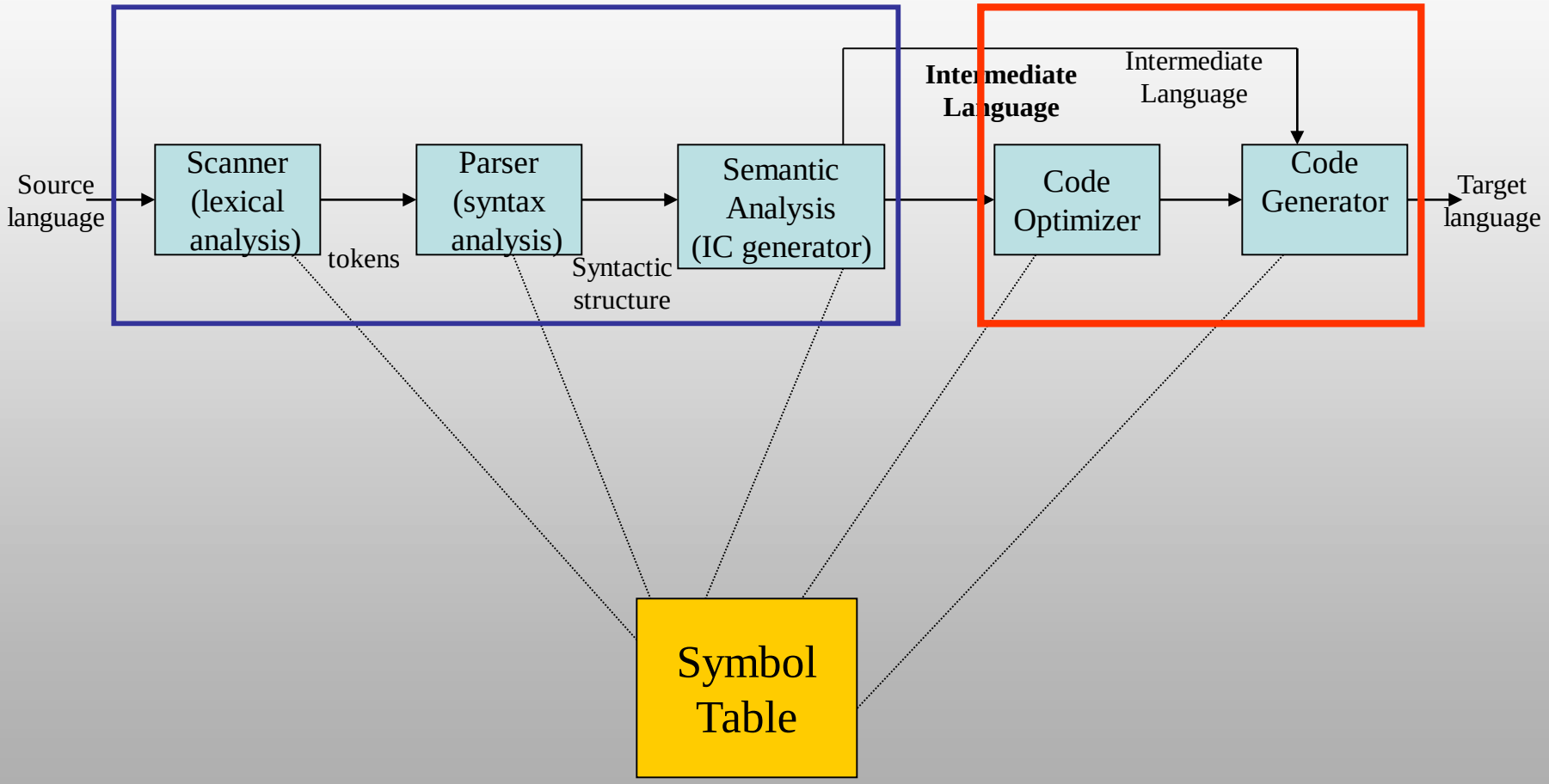
Compiler Architecture

In more detail:



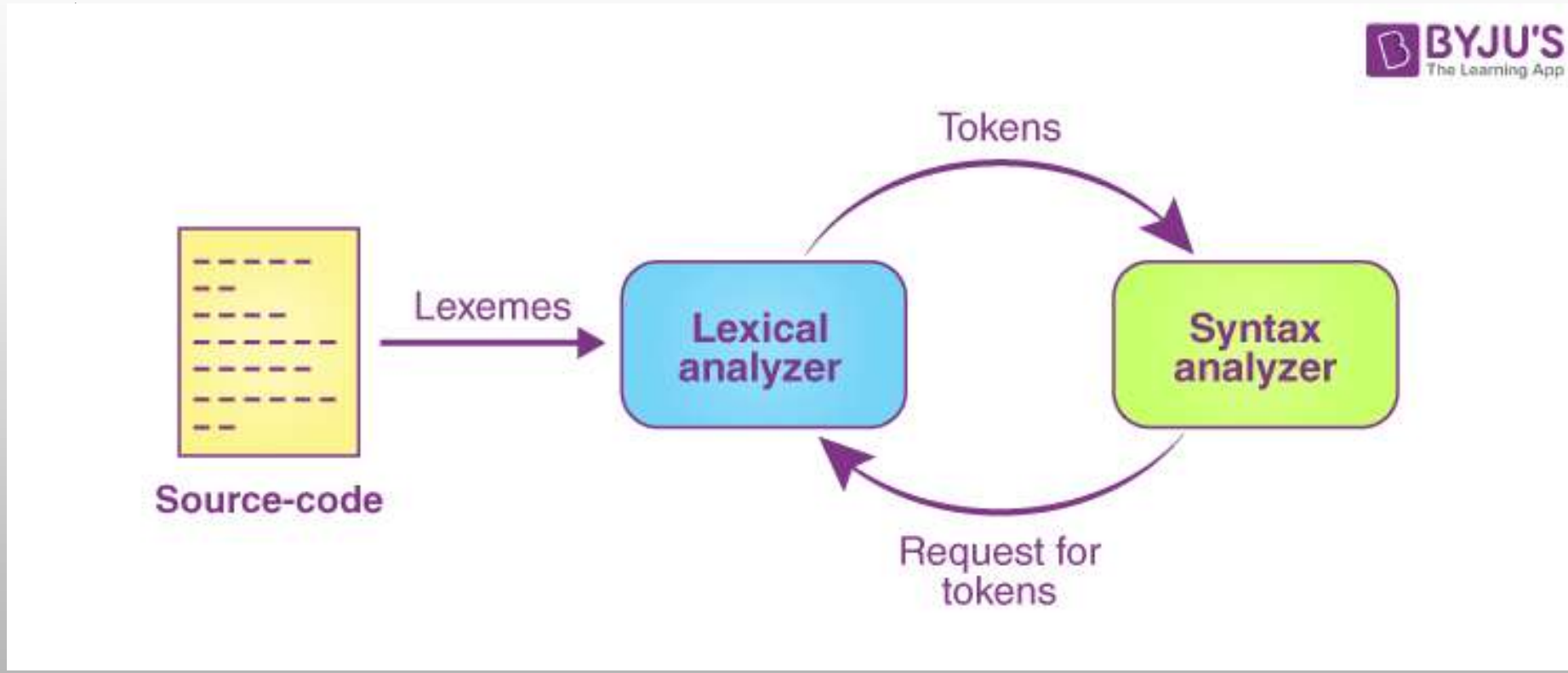
- Separation of Concerns
- Retargeting

Compiler Architecture



Lexical Analysis

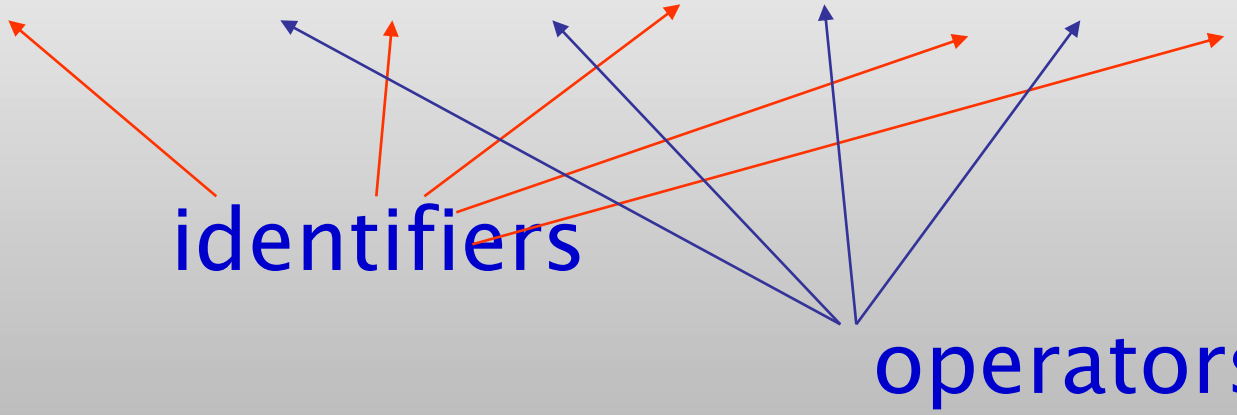
- Lexical analysis, or scanning, is a fundamental step in NLP.
- In programming languages, this process involves the lexical analyzer (lexer or scanner) reading the source code character by character to group these characters into tokens, the smallest units in the code that convey meaning.


BYJU'S
The Learning App
Tokens
Lexemes
Lexical analyzer
Syntax analyzer
Source-code
Request for tokens

Input: result = a + b * c / d

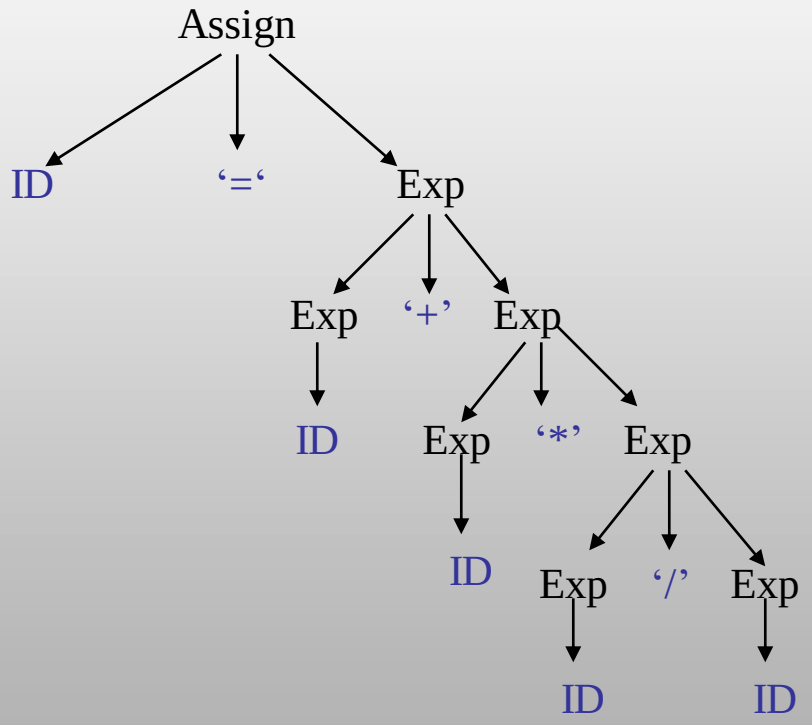
- Tokens:

  ‘result’, ‘=‘, ‘a’, ‘+’, ‘b’, ‘*’, ‘c’, ‘/’, ‘d’



Input: result = a + b * c / d

```
Exp    ::=  Exp '+' Exp
          |   Exp '-' Exp
          |   Exp '*' Exp
          |   Exp '/' Exp
          |   ID
Assign ::=   ID '=' Exp
```



How Lexical Analyzer Works?

- Input preprocessing
- Tokenization
- Token classification
- Token validation

Find valid tokens

```
int main()
{
   // 2 variables
   int a, b;
   a = 10;
  return 0;
}
```

- int main()
- {
- int a = 10, b = 20;
- printf("sum is:%d",a+b);
- return 0;
- }
- Answer: Total number of token: 27.

Lexical Analysis

- A scanner groups input characters into tokens

input: x = x * (acc+123)

token	value
identifier	x
equal	=
identifier	x
star	*
left-paren	(
identifier	acc
plus	+
integer	123
right-paren	)

- Tokens are typically represented by numbers

Free form vs Fixed form

- Free form languages (all modern ones)
  - White space does not matter. Ignore these:
    - Tabs, spaces, new lines, carriage returns
  - Only the ordering of tokens is important
- Fixed format languages (historical)
  - Layout is critical
    - Fortran, label in cols 1-6
    - COBOL, area A B
    - Lexical analyzer must know about layout to find tokens

Typical Tokens in Programming Languages

- Operators & Punctuation
  - + - * / ( ) { } [ ] ; : :: < <= == = != ! …
  - Each of these is a distinct lexical class
- Keywords
  - if while for goto return switch void …
  - Each of these is also a distinct lexical class (*not* a string)
- Identifiers
  - A signle ID lexical class, but parameterized by actual id
- Integer constants
  - A single INT lexical class, but parameterized by int value
- Other constants, etc

Lexical Analysis

- Lexical analyzer splits it into tokens
  - Token = sequence of characters (symbolic name) representing a single terminal symbol
- Identifiers: myVariable …
- Literals: 123 5.67 true …
- Keywords: char sizeof …
- Operators: + − * / …
- Punctuation: ; , } { …
- Discards whitespace and comments

Tasks of a Scanner

- A typical scanner:
  - recognizes the keywords of the language
    - these are the reserved words that have a special meaning in the language, such as the word `class` in Java
  - recognizes special characters, such as (and), or groups of special characters, such as := and ==
  - recognizes identifiers, integers, reals, decimals, strings, etc
  - ignores whitespaces (tabs, blanks, etc) and comments
  - recognizes and processes special directives (such as the `#include "file"` directive in C)

Example of tokens

- Keywords; Examples – for, while, if etc.
- Identifier; Examples – Variable name, function name, etc.
- Operators; Examples '+', '++', '–' etc.
- Separators; Examples ',' ';' etc

- Example of Non-Tokens
- Comments, preprocessor directive, macros, blanks, tabs, newline, etc

Examples of Tokens in C

Tokens	Lexemes
identifier	Age, grade,Temp, zone, q1
number	3.1416, -498127,987.76412097
string	“A cat sat on a mat.”, “90183654”
open parentheses	(
close parentheses	)
Semicolon	;
reserved word if	IF, if, If, iF

Reserved Keywords in C

- auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, wchar_t, while
- C++ added a bunch: bool, catch, class, dynamic_cast, inline, private, protected, public, static_cast, template, this, virtual and others
- Each keyword is mapped to its own token

Whitespace

Whitespace is any space, tab, end-of-line character (or characters), or character sequence inside a comment

No token may contain embedded whitespace (unless it is a character or string literal)

Example:

`>=` one token

`> =` two tokens

Lexical errors

- Some errors are out of power of lexical analyzer to recognize:
  - fi (a == f(x)) …
- However it may be able to recognize errors like:
  - d = 2r
- Such errors are recognized when no pattern for tokens matches a character sequence

Lexical Errors vs. Other Errors

- **Lexical** error: a character sequence cannot be matched to a valid token under the language rules.
- **Syntax** error: valid tokens appear in an invalid grammatical order, such as a missing closing parenthesis.
- **Semantic** error: the structure is grammatical but violates language rules, such as incompatible types.
- **Security** warning: suspicious Unicode lookalikes may need a separate policy or analysis layer.
- A lexer should report what it can establish; it should not claim to prove program intent

```
#include <iostream>
using namespace std;

int main() {

    printf("Geeksforgeeks");$
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
/* comment
    cout<<"GFG!";
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {

    int 3num= 1234;    /* spelling error as identifier
                          cannot start with a number*/
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {

    int x = 12$34; /*lexical error as '$' doesn't
                    belong within 0-9 range*/
    return 0;
}
```

```
#include <iostream> /*missing 'o'
character
                        hence lexical error*/
using namespace std;

int main() {

    cout<<"GFG!";
    return 0;
}
```

```
Void main()
{
    int x=10, y=20;
    char * a;
    a= &x;
    x= 1xab;
}
```

Redefining Identifiers can be dangerous

```
program confusing;
const true = false;
begin
 if  (a<b) = true then
 f(a)
 else …
```

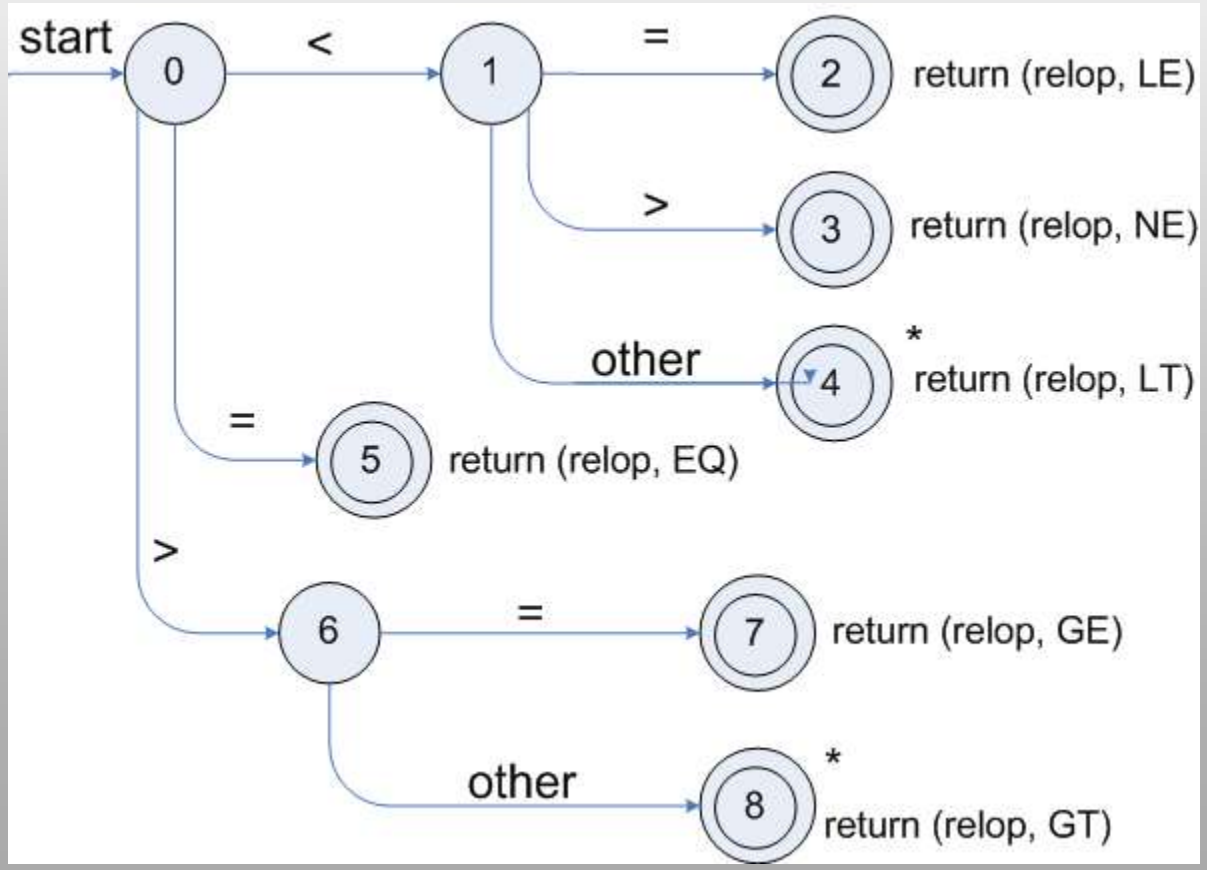
Sentinels

$$E = M_{eof} * C * * 2_{eof}$$ eof

```
Switch (*forward++) {
    case eof:
        if (forward is at end of first buffer) {
            reload second buffer;
            forward = beginning of second buffer;
        }
        else if {forward is at end of second buffer) {
            reload first buffer;\
            forward = beginning of first buffer;
        }
        else /* eof within a buffer marks the end of input */
            terminate lexical analysis;
        break;
    cases for the other characters;
}
```

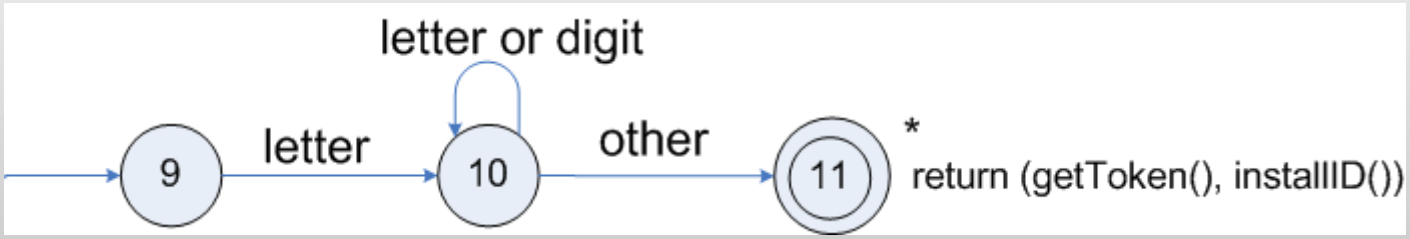
Transition diagrams

- Transition diagram for relop



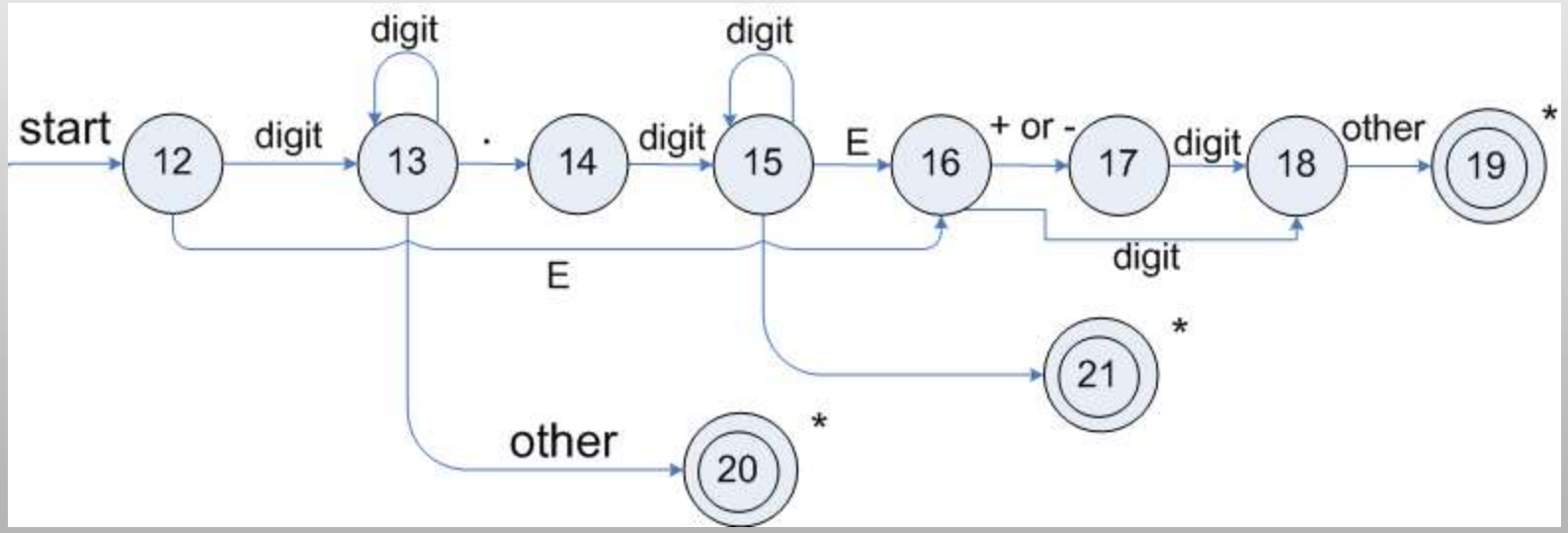
Transition diagrams (cont.)

- Transition diagram for reserved words and identifiers



Transition diagrams (cont.)

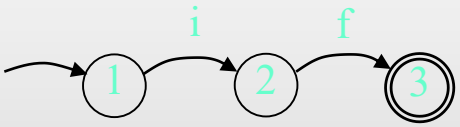
- Transition diagram for unsigned numbers



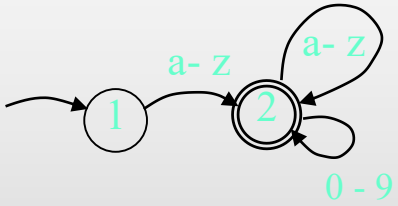
Recognition

- state = 0;
- while ((c = next_char()) != EOF) {
  - switch (state) {
    - case 0: if (c == 'a') state = 1;
      - break;
    - case 1: if (c == 'b') state = 2;
      - break;
    - case 2: if (c == 'c') state = 3;
      - break;
    - case 3: if (c == 'a') state = 1;
      - else { ungetchar(); return (TRUE); }
      - break;
    - default:
      - error();
  - }
- }
- if (state == 3) return (TRUE) else return (FALSE);

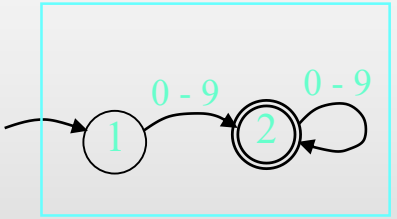
Finite Automata for the Lexical Tokens



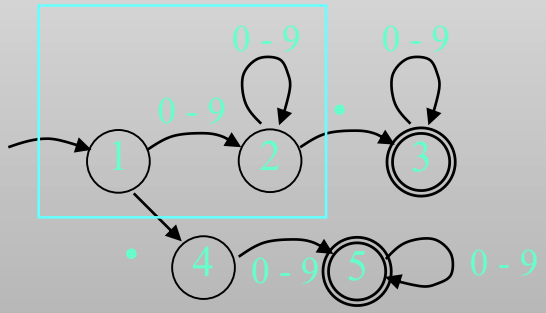
IF



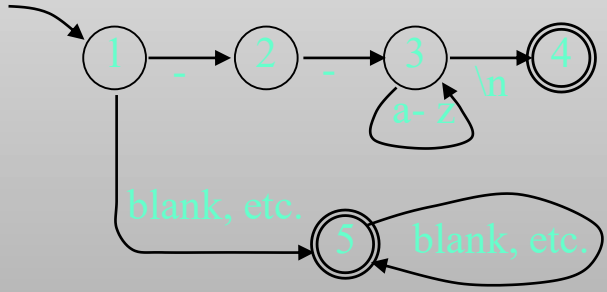
ID



NUM



REAL



White space
(and comment starting with '- -')



error

(Appel, pp. 21)

LEXICAL ANALYSIS

Lexical Errors

- Deleting an extraneous character
- Inserting a missing character
- Replacing an incorrect character by a correct character
- Transposing two adjacent characters(such as , fi=>if)
- Pre-scanning

Tokens / Patterns / Regular Expressions

Lexical Analysis - searches for matches of lexeme to pattern

Lexical Analyzer returns:<actual lexeme, symbolic identifier of token>

For Example:

Token	Symbolic ID
`if`	1
`then`	2
`else`	3
`>,>=,<,…`	4
`:=`	5
id	6
int	7
real	8

Set of all regular expressions plus symbolic ids plus analyzer define required functionality.

REs ---algs---> NFA ---algs---> DFA (program for simulation)

Lexical Analysis using AI

AI extension: use large language models (LLMs) to propose tests and explain failures—not to replace the language specification.

- Learning outcome: build a deterministic lexer, evaluate it with test cases, and critically assess AI assistance.

Terminology: Do Not Confuse These Concepts

- Lexeme: the actual character sequence in the source, e.g., count or >=.
- Token type: the category assigned to a lexeme, e.g., IDENTIFIER or REL_OP.
- Token instance: typically (type, lexeme/value, line, column).
- Pattern: the rule that describes a token class, often a regular expression.
- LLM subword token: a unit used by a language model tokenizer; it is not the same as a compiler token.

Example: Token Stream and Longest Match

Input: total >= 100 && ready == true

```
IDENTIFIER("total")
REL_OP(">=")
INTEGER_LITERAL(100)
LOGICAL_AND("&&")
IDENTIFIER("ready")
EQUALITY_OP("==")
BOOLEAN_LITERAL(true)
```

Practical Exercise: AI and Test Cases

Rule: `[A-Za-z_][A-Za-z0-9_]*`

```
count       -> ?
_count2     -> ?
2count      -> ?
user-name   -> ?
class       -> identifier or keyword?
naïve       -> valid under this rule?
```

Follow-up: if Unicode identifiers are supported, revise the rule; keywords need separate handling.

Compiler lexical analysis vs. NLP tokenization

int total = 100 + count;

- KEYWORD(int)
- IDENTIFIER(total)
- ASSIGNMENT(=)
- INTEGER(100)
- PLUS(+)
- IDENTIFIER(count)
- SEMICOLON(;)

Compiler lexical analysis vs. NLP tokenization

Artificial intelligence is transforming healthcare.

- ["Artificial", "intelligence", "is",
- "transforming", "healthcare", "."]